



Aplicação de Lista de Compras com Preços e Alertas

SHOPISEL

Orientador:

Doutor Fernando Miguel Gamboa de Carvalho

Júri:

Presidente:

Vogais:

Doutor

Doutor

Julho de 2026

Aplicação de Lista de Compras com Preços e Alertas

SHOPISEL

Orientador:

Doutor Fernando Miguel Gamboa de Carvalho, IPL/ISEL

Júri:

Presidente:

Vogais:

Julho de 2026

Agradecimentos

Gostaríamos de expressar o nosso sincero agradecimento ao Doutor Fernando Miguel Gamboa de Carvalho pela sua disponibilidade constante e pela rapidez nas respostas ao longo do desenvolvimento deste projeto, contributos esses que se revelaram fundamentais para a sua concretização.

Agradecemos igualmente ao João Pereira, ao Pedro Tainha e ao Breno Nico pela idealização do projeto, bem como por todas as explicações, esclarecimentos e orientações fornecidas, que foram essenciais para a definição da estrutura, abordagem e formas de pensar adotadas ao longo deste trabalho.

A todos, o nosso muito obrigado pelo apoio e colaboração.

Declaração de integridade

Declaro que este trabalho de projeto é o resultado da minha investigação pessoal e independente. O seu conteúdo é original e todas as fontes listadas nas referências bibliográficas foram consultadas e estão devidamente mencionadas no texto. Mais declaro que todas as referências científicas e técnicas relevantes para o desenvolvimento do trabalho estão devidamente citadas e constam das referências bibliográficas.

O autor

Lisboa, 20 de Julho de 2026

Abstract

This project presents the development of a mobile application for managing shopping lists enhanced with price comparison and alert functionalities, supported by a scalable microservices-based backend architecture. The solution is designed to be used in real-world contexts, such as supermarkets, allowing users to efficiently create and manage shopping lists, mark items during purchases, and access updated product prices across different stores and locations.

The system integrates multiple technologies, including a React Native mobile application, a React-based web application for backoffice management, and a set of RESTful backend services developed in .NET. Authentication and authorization are handled através de Keycloak, while push notifications are delivered through Firebase Cloud Messaging to inform users about price changes and list updates.

A central data management component is responsible for handling products, prices, and store information, ensuring consistency and reliability. The architecture follows a modular and scalable approach, enabling the integration of additional services and supporting high performance and data synchronization requirements.

Additionally, the project explores the concept of White Label development, allowing the customization of applications through dedicated CI/CD pipelines. The use of AI-powered tools, such as GitHub Copilot, is also incorporated to support development and validation processes.

Overall, this work demonstrates how a seemingly simple application can evolve into a complete digital ecosystem, addressing challenges related to distributed systems, data management, and real-time user interaction.

Resumo

O presente projeto apresenta o desenvolvimento de uma aplicação móvel para gestão de listas de compras, enriquecida com funcionalidades de comparação de preços e definição de alertas, suportada por uma arquitetura de backend baseada em microserviços. A solução foi concebida para utilização em contextos reais, como supermercados, permitindo aos utilizadores criar e gerir listas de compras de forma eficiente, marcar itens durante a compra e consultar preços atualizados de produtos em diferentes lojas e localizações.

O sistema integra diversas tecnologias, incluindo uma aplicação móvel desenvolvida em React Native, uma aplicação web em React para backoffice e um conjunto de serviços backend com APIs REST implementados em .NET. A autenticação e autorização são asseguradas através do Keycloak, enquanto as notificações são geridas com recurso ao Firebase Cloud Messaging, permitindo alertar os utilizadores para alterações de preços e atualizações das listas.

A solução inclui ainda um mecanismo centralizado de gestão de dados, responsável pela manutenção de informação relativa a produtos, preços e lojas, garantindo consistência e fiabilidade. A arquitetura adotada segue princípios modulares e escaláveis, possibilitando a evolução do sistema e a integração de novos serviços.

Adicionalmente, o projeto explora o conceito de desenvolvimento White Label, permitindo a customização das aplicações através de pipelines dedicados de CI/CD. Foi também utilizada inteligência artificial, nomeadamente ferramentas baseadas em LLMs como o GitHub Copilot, como apoio ao desenvolvimento e validação do sistema.

Em suma, este trabalho demonstra como uma aplicação aparentemente simples pode evoluir para um ecossistema digital completo, abordando desafios relacionados com sistemas distribuídos, gestão de dados e interação em tempo real com o utilizador.

Contents

Agradecimentos	i
Declaração de integridade	iii
Abstract	v
Resumo	vii
Acronyms	xv
1 Introduction	1
1.1 Context and motivation	1
1.1.1 Scope of the Problem	2
1.2 Approach	2
1.3 Objetivos	3
1.4 Estrutura do Relatório	4
2 State-Of-The-Art	5
2.1 Plataformas digitais para apoio a compras	5
2.2 Arquitetura orientada a serviços	5
2.3 Persistência de dados e modelo relacional	6
2.4 Aplicações web modernas	6
2.5 Aplicações móveis multiplataforma	7
2.6 Autenticação e autorização	7
2.7 Recolha automática de dados	7
2.8 Containerização, CI/CD e deployment	8
2.9 Síntese	8
3 Trabalho Relacionado	9
3.1 Aplicações de gestão de listas de compras	9
3.2 Plataformas de comparação de preços	10
3.3 Aplicações próprias de cadeias de supermercados	10
3.4 Análise comparativa e posicionamento do ShopISEL	11
3.5 Síntese	11

4	Trabalho Desenvolvido	13
4.1	Visão Geral da Arquitetura	13
4.2	Serviços de Backend	14
4.2.1	List Service	15
4.2.2	Products Service	15
4.2.3	Prices Service	16
4.2.4	Stores Service	16
4.2.5	Account Service	16
4.2.6	MatchQueue Service	17
4.3	Pipeline de Recolha de Dados	17
4.3.1	Scraper Continente	18
4.3.2	Scraper Pingo Doce	19
4.3.3	Scraper Orchestrator	19
4.4	FCM Notification Worker	19
4.5	Aplicação Web	20
4.6	Aplicação Móvel	20
4.7	Autenticação e Autorização	21
4.8	Gateway e Infraestrutura de Deployment	22
5	Processo de Desenvolvimento e Integração Contínua	23
5.1	Organização do Projeto em Repositórios	23
5.2	Controlo de Versões e Estratégia de Branches	24
5.3	Pipelines de Integração e Entrega Contínua	24
5.4	Reusable Workflows e Orquestração de Scrapers	25
5.5	Ferramentas de Suporte ao Desenvolvimento	25
5.6	Gestão de Configuração e Segredos	26
6	Testes e Avaliação	27
6.1	Estratégia de Testes	27
6.2	Infraestrutura de Testes	27
6.2.1	Base de dados em memória com SQLite	28
6.2.2	Handler de autenticação de teste	28
6.3	Casos de Teste do List Service	28
6.3.1	Fluxo CRUD completo	28
6.3.2	Isolamento por proprietário	29
6.3.3	Filtragem no GET /lists	29
6.4	Execução dos Testes nas Pipelines de CI/CD	29
6.5	Avaliação Funcional	29
7	Conclusão	31
7.1	Síntese do Trabalho Desenvolvido	31
7.2	Desafios Encontrados	32

7.3	Trabalho Futuro	33
7.4	Considerações Finais	33

List of Figures

4.1	Visão geral da arquitetura do sistema ShopISEL	14
4.2	Pipeline de recolha de dados e entrega de notificações	18
5.1	Fluxo de Continuous Integration/Continuous Delivery (CI/CD) para os serviços de backend	25

List of Tables

3.1	Comparação entre soluções existentes e o ShopISEL	11
-----	---	----

Acronyms

API	Application Programming Interface
CI/CD	Continuous Integration/Continuous Delivery
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
OIDC	OpenID Connect
REST	Representational State Transfer
URL	Uniform Resource Locator
FCM	Firebase Cloud Messaging

Chapter 1

Introduction

O planeamento de compras continua a exigir que os consumidores consultem múltiplas fontes de informação para comparar preços, verificar promoções ou organizar listas de produtos. Este processo torna-se frequentemente demorado e ineficiente, especialmente quando envolve várias lojas ou supermercados.

Este projeto propõe o desenvolvimento de uma plataforma digital que centraliza informação de produtos e preços, permitindo aos utilizadores criar listas de compras inteligentes, comparar preços entre diferentes lojas e receber alertas quando ocorrem alterações relevantes.

A solução será disponibilizada através de duas interfaces principais: uma aplicação móvel e uma aplicação web.

A aplicação móvel será utilizada no contexto real de compras e permitirá aos utilizadores criar e gerir listas de forma simples, consultar e comparar preços entre diferentes lojas e visualizar preços próximos com base na localização do utilizador. Adicionalmente, os utilizadores poderão receber notificações quando um produto sofrer uma redução de preço e identificar rapidamente produtos através da leitura de códigos de barras ou códigos QR com a câmara do dispositivo.

A aplicação web permitirá uma preparação mais confortável das listas de compras num computador, bem como a consulta de comparações de preços entre diferentes lojas e localizações, oferecendo uma visão mais abrangente da informação disponível.

Um dos elementos centrais da plataforma será a georreferenciação de preços. O mesmo produto poderá ter valores diferentes consoante a loja e a localização geográfica, permitindo ao sistema apresentar ao utilizador as opções mais vantajosas com base na sua proximidade. Desta forma, a aplicação pretende simplificar o processo de planeamento de compras e ajudar os utilizadores a tomar decisões mais informadas, poupando tempo e dinheiro.

1.1 Context and motivation

Nos últimos anos, a necessidade de otimizar o processo de planeamento de compras tem vindo a aumentar, impulsionada pela crescente diversidade de produtos, lojas e promoções disponíveis. Atualmente, os consumidores são frequentemente obrigados a consultar múlti-

plas fontes de informação para comparar preços, verificar campanhas promocionais e organizar listas de compras, o que torna este processo moroso e pouco eficiente. Esta realidade é particularmente evidente quando envolve diferentes superfícies comerciais, como supermercados e lojas físicas, onde os preços podem variar significativamente.

Este projeto foca-se especificamente no desenvolvimento de uma plataforma digital integrada que permita centralizar informação de produtos e preços, simplificando o processo de tomada de decisão do utilizador. A solução proposta combina uma aplicação móvel e uma aplicação web, permitindo não só a criação e gestão de listas de compras, mas também a comparação de preços entre diferentes lojas e localizações. Adicionalmente, são introduzidas funcionalidades como alertas de alteração de preços, leitura de códigos de barras e georreferenciação, proporcionando uma experiência mais inteligente e adaptada ao contexto real de compra.

1.1.1 Scope of the Problem

A gestão eficiente de compras apresenta vários desafios que vão além da simples criação de listas de produtos. Um dos principais problemas reside na dispersão da informação, uma vez que os preços dos produtos variam consoante a loja e a localização geográfica, obrigando o utilizador a realizar múltiplas consultas para encontrar a melhor opção. Esta dificuldade é reforçada pelo facto de muitas soluções existentes se focarem apenas em contextos de comércio eletrónico, não respondendo adequadamente às necessidades de compras em lojas físicas.

Outro desafio relevante prende-se com a falta de integração entre funcionalidades. Aplicações atuais tendem a oferecer apenas gestão de listas ou comparação de preços de forma isolada, não existindo uma solução unificada que combine ambas de forma eficiente. Para além disso, a inexistência de mecanismos automáticos de alerta para variações de preço limita a capacidade do utilizador de aproveitar oportunidades de poupança em tempo útil.

Neste contexto, o problema a abordar passa pela criação de um sistema que integre, de forma coerente e escalável, a gestão de produtos, preços e lojas, garantindo simultaneamente desempenho, consistência de dados e relevância da informação apresentada ao utilizador. A introdução de georreferenciação e notificações em tempo real surge como elemento diferenciador, permitindo apresentar opções mais vantajosas com base na proximidade e nas preferências do utilizador.

1.2 Approach

Para alcançar os objetivos definidos, foi concebido e desenvolvido um sistema distribuído composto por uma aplicação móvel, uma aplicação web e uma infraestrutura de backend, recorrendo a tecnologias modernas amplamente adotadas, de forma a garantir usabilidade, escalabilidade e facilidade de manutenção.

A solução proposta segue uma arquitetura orientada a serviços, na qual o backend se

encontra dividido em vários serviços responsáveis por domínios específicos, como produtos, preços, lojas e utilizadores. Estes serviços são disponibilizados através de APIs REST, permitindo uma comunicação eficiente entre os diferentes componentes e promovendo uma clara separação de responsabilidades.

A aplicação móvel, desenvolvida com recurso a React Native, foi concebida para utilização em contexto real de compra, disponibilizando funcionalidades como gestão de listas, comparação de preços, leitura de códigos de barras e receção de notificações em tempo real. Por sua vez, a aplicação web, desenvolvida em React, oferece uma interface mais completa para preparação e análise das compras, tirando partido de um ambiente de utilização mais confortável.

A gestão de dados é assegurada através de uma base de dados relacional, garantindo integridade e eficiência nas operações de consulta, particularmente em cenários onde um mesmo produto pode apresentar preços distintos consoante a loja e a localização. A incorporação de mecanismos de georreferenciação permite ao sistema apresentar informação contextualizada, baseada na proximidade do utilizador.

Adicionalmente, o sistema integra um mecanismo de notificações push, possibilitando o envio de alertas relativos a alterações de preços, contribuindo para uma maior proatividade na tomada de decisão por parte do utilizador. O processo de desenvolvimento é suportado por pipelines de CI/CD, que permitem automatizar tarefas de construção, teste e deploy, assegurando a qualidade e consistência do software.

Em conjunto, esta abordagem permite o desenvolvimento de uma solução robusta, escalável e centrada no utilizador, capaz de responder de forma eficaz aos desafios identificados no domínio do planeamento de compras.

1.3 Objetivos

Os objetivos principais do projeto ShopISEL são:

1. Desenvolver uma plataforma integrada que permita ao utilizador criar e gerir listas de compras, consultar preços de produtos em múltiplas lojas e receber alertas de descida de preço;
2. Implementar um sistema de recolha automática de dados de produtos e preços de insígnias de retalho portuguesas, sem dependência de APIs proprietárias;
3. Disponibilizar a plataforma através de uma aplicação móvel (Android e iOS) e de uma aplicação web, com experiências de utilização adequadas a cada contexto;
4. Integrar georreferenciação de lojas, permitindo ao utilizador identificar as opções mais vantajosas na sua proximidade geográfica;
5. Garantir a segurança e privacidade dos dados do utilizador através de um sistema de autenticação centralizado baseado em padrões abertos (OIDC / OAuth 2.0);

6. Assegurar a qualidade e a entrega contínua do sistema através de pipelines de CI/CD automatizadas.

1.4 Estrutura do Relatório

O presente relatório está organizado da seguinte forma:

- **Capítulo 2 — Estado da Arte:** analisa as tecnologias, frameworks e ferramentas mais relevantes para o desenvolvimento da plataforma, justificando as escolhas técnicas efetuadas;
- **Capítulo 3 — Trabalho Relacionado:** apresenta e compara soluções existentes no domínio das aplicações de compras e comparação de preços, identificando o posicionamento do ShopISEL;
- **Capítulo 4 — Trabalho Desenvolvido:** descreve em detalhe a arquitetura, os componentes e a implementação de todos os elementos da plataforma;
- **Capítulo 5 — Processo de Desenvolvimento e Integração Contínua:** descreve o processo de desenvolvimento adotado, a organização dos repositórios e as pipelines de CI/CD;
- **Capítulo 6 — Testes e Avaliação:** apresenta a estratégia de testes, os casos de teste implementados e a avaliação funcional do sistema;
- **Capítulo 7 — Conclusão:** sintetiza o trabalho realizado, reflete sobre os desafios encontrados e identifica direções de evolução futura.

Chapter 2

State-Of-The-Art

Este capítulo apresenta o estado da arte das tecnologias, metodologias e ferramentas mais relevantes para o desenvolvimento da plataforma ShopISEL. A análise é orientada pelas necessidades concretas do projeto: disponibilizar uma solução composta por aplicação web, aplicação móvel, serviços de backend, persistência relacional, autenticação centralizada, recolha automática de dados de produtos e mecanismos de entrega contínua.

2.1 Plataformas digitais para apoio a compras

As soluções digitais para apoio a compras evoluíram de listas simples, mantidas localmente pelo utilizador, para plataformas integradas capazes de combinar catálogos de produtos, preços, histórico de alterações, alertas e informação contextual. No domínio dos supermercados, esta evolução é particularmente relevante, uma vez que o preço de um mesmo produto pode variar consoante a loja, a campanha promocional, a disponibilidade e a localização geográfica.

No contexto deste projeto, a plataforma não se limita a representar uma lista de compras. A solução pretende relacionar produtos, categorias, lojas, preços e utilizadores, permitindo que a lista seja um ponto de entrada para decisões de compra mais informadas. Esta abordagem aproxima a aplicação de sistemas de comparação e monitorização de preços, mas mantém o foco numa utilização prática e recorrente: preparar compras, consultar alternativas e acompanhar alterações relevantes ao longo do tempo.

2.2 Arquitetura orientada a serviços

Uma tendência consolidada no desenvolvimento de plataformas distribuídas é a separação do sistema em serviços especializados. Esta abordagem permite dividir responsabilidades por domínio funcional, reduzir o acoplamento entre componentes e facilitar a evolução independente de cada parte do sistema. No projeto ShopISEL, esta separação encontra-se refletida em serviços distintos para listas de compras, produtos, lojas e preços.

A utilização de Application Programming Interface (API)s Representational State Transfer (REST) continua a ser uma escolha frequente em sistemas web e móveis, devido à sua

simplicidade, interoperabilidade e boa integração com clientes baseados em HyperText Transfer Protocol (HTTP). Estudos recentes sobre frameworks para serviços web destacam que a escolha da tecnologia de backend deve considerar desempenho, escalabilidade, produtividade e maturidade do ecossistema [?]. No caso do ShopISEL, estes critérios justificam a adoção de serviços em ASP.NET Core, com endpoints organizados por domínio e contratos explícitos para a comunicação com as aplicações cliente.

O projeto utiliza Minimal APIs em ASP.NET Core, uma abordagem recomendada para a criação de APIs HTTP com menor quantidade de código estrutural e boa adequação a serviços pequenos ou focados num domínio específico [?]. Esta opção é coerente com a divisão do repositório, onde cada serviço encapsula o seu modelo de dados, endpoints, lógica de negócio e configuração de execução.

2.3 Persistência de dados e modelo relacional

A gestão de produtos, lojas, listas e preços exige consistência entre entidades relacionadas. Por esse motivo, as bases de dados relacionais continuam a ser uma solução adequada para sistemas onde existem relações fortes, restrições de integridade e necessidade de consultas estruturadas. PostgreSQL é uma das soluções relacionais mais utilizadas em aplicações modernas, sendo mantida com documentação oficial e suporte para versões atuais [?].

No ShopISEL, os serviços de backend recorrem a Entity Framework Core com o provider Npgsql para PostgreSQL. Esta combinação permite modelar entidades em C#, aplicar migrações e interagir com a base de dados através de uma camada de acesso a dados integrada no ecossistema .NET [?]. A utilização de migrações é particularmente importante porque o sistema evolui por serviços: cada domínio pode alterar o seu esquema de dados sem exigir uma reorganização global da aplicação.

Esta opção também facilita o suporte a cenários relevantes para o projeto, como a associação de itens a listas de compras, a categorização de produtos, o registo de lojas e a manutenção de informação de preços recolhida de fontes externas. A persistência relacional oferece, assim, uma base estável para relacionar dados operacionais com funcionalidades de comparação e consulta.

2.4 Aplicações web modernas

As aplicações web modernas privilegiam interfaces reativas, comunicação assíncrona com APIs e componentes reutilizáveis. React consolidou-se como uma biblioteca amplamente adotada para construção de interfaces declarativas, tirando partido de componentes e hooks para gerir estado, efeitos e composição de comportamentos [?]. Vite, por sua vez, é frequentemente utilizado em projetos frontend por simplificar a configuração e acelerar o ciclo de desenvolvimento.

No ShopISEL, a aplicação web é construída com React, TypeScript e Vite. Esta combi-

nação permite desenvolver uma interface orientada a componentes para consultar produtos, gerir listas, visualizar alertas e interagir com os serviços de backend. O uso de TypeScript acrescenta verificação estática ao código cliente, reduzindo erros em estruturas de dados trocadas com as APIs. A presença de bibliotecas de componentes e utilitários de interface, como Material UI, Radix UI e Tailwind, permite acelerar a construção de ecrãs consistentes sem abandonar a personalização visual da plataforma.

2.5 Aplicações móveis multiplataforma

No contexto de compras presenciais, a aplicação móvel é essencial porque acompanha o utilizador no local onde a decisão de compra acontece. Funcionalidades como consulta rápida de listas, leitura de códigos de barras, comparação de preços e receção de alertas dependem de uma experiência adaptada ao dispositivo móvel.

React Native permite desenvolver aplicações móveis com uma base tecnológica próxima do ecossistema React, enquanto Expo fornece ferramentas, módulos nativos e serviços que simplificam a criação de aplicações para Android, iOS e web a partir de um projeto JavaScript/TypeScript [?]. No repositório do ShopISEL, a aplicação móvel usa Expo, Expo Router, React Native, NativeWind e módulos como Expo Camera e Expo Secure Store. Esta escolha é adequada ao projeto porque permite integrar navegação por separadores, autenticação, armazenamento seguro e leitura por câmara, mantendo uma base de desenvolvimento coerente com a aplicação web.

2.6 Autenticação e autorização

Sistemas que manipulam listas pessoais e preferências de utilizador necessitam de autenticação fiável e de uma separação clara entre identidade e lógica aplicacional. Keycloak é uma solução de gestão de identidade e acesso que suporta protocolos como OpenID Connect e OAuth 2.0, permitindo proteger aplicações e serviços através de tokens e endpoints padronizados [?].

No ShopISEL, Keycloak é utilizado como componente central de autenticação. O serviço de listas valida tokens JSON Web Token (JWT) emitidos por Keycloak, verificando emissor, validade e partes autorizadas antes de permitir o acesso a recursos protegidos. Esta abordagem evita que cada serviço implemente autenticação própria e permite que web, mobile e backend partilhem um modelo comum de identidade.

2.7 Recolha automática de dados

Uma plataforma de comparação de preços depende da qualidade e atualização dos dados. Quando não existe uma API pública uniforme para todas as lojas, a recolha automática por

scraping torna-se uma alternativa prática, embora exija cuidados com estrutura das páginas, normalização de dados e robustez perante alterações externas.

O projeto inclui scrapers específicos para Continente e Pingo Doce. Estes componentes recolhem produtos por categoria, extraem informação como nome, preço, promoção, imagem e disponibilidade, e podem sincronizar os resultados com PostgreSQL através da variável de ambiente `PRODUCTS_DB_CONNECTION`. No caso do Pingo Doce, a utilização de Playwright é adequada a páginas dinâmicas, uma vez que esta ferramenta permite automatizar browsers e interagir com conteúdo renderizado no cliente. A existência de ficheiros JavaScript Object Notation (JSON) de saída também facilita a validação dos dados extraídos e a análise posterior por categoria.

2.8 Containerização, CI/CD e deployment

A entrega de aplicações compostas por vários serviços exige mecanismos que garantam reprodutibilidade e automatização. Docker permite empacotar aplicações e dependências em containers, tornando mais previsível a execução em ambientes distintos [?]. Esta característica é especialmente relevante no ShopISEL, onde coexistem serviços .NET, frontend web, gateway Nginx e componentes de suporte.

O repositório inclui Dockerfiles e workflows de GitHub Actions para automatizar validação, construção e publicação de imagens. GitHub Actions suporta a automatização de workflows diretamente no repositório, incluindo tarefas de integração e entrega contínua [?]. A configuração de deployment usa Render, com serviços baseados em imagens e um Nginx configurado como ponto de entrada para encaminhar pedidos para frontend, listas, produtos, lojas e preços. A documentação do Render suporta este modelo de deployment com imagens Docker pré-construídas ou Dockerfiles do repositório [?].

2.9 Síntese

O estado da arte analisado mostra que a arquitetura escolhida para o ShopISEL está alinhada com práticas atuais de desenvolvimento de aplicações distribuídas. A separação em serviços facilita a evolução por domínio, PostgreSQL e Entity Framework Core oferecem uma base consistente para persistência, React e Expo permitem cobrir web e mobile com tecnologias próximas, Keycloak centraliza autenticação e Docker com GitHub Actions apoia a entrega contínua.

Estas escolhas tecnológicas respondem diretamente aos objetivos do projeto: centralizar informação de produtos e preços, suportar listas de compras inteligentes, permitir comparação entre lojas e preparar a plataforma para evolução futura com novas fontes de dados, novas funcionalidades de alerta e melhorias na experiência do utilizador.

Chapter 3

Trabalho Relacionado

Este capítulo analisa soluções existentes no domínio das aplicações de listas de compras e comparação de preços, com o objetivo de identificar as limitações das abordagens atuais e justificar as opções tomadas no desenvolvimento do ShopISEL. A análise divide-se em três categorias principais: aplicações de gestão de listas de compras, plataformas de comparação de preços e aplicações próprias de cadeias de supermercados.

3.1 Aplicações de gestão de listas de compras

O mercado de aplicações de listas de compras é relativamente maduro, com diversas soluções estabelecidas que oferecem funcionalidades base de criação e partilha de listas.

O **Bring!** é uma das aplicações mais populares na Europa para gestão de listas de compras. Permite criar listas colaborativas, partilhá-las com membros da família ou amigos em tempo real, e sugere produtos com base no histórico de compras. A interface é intuitiva e suporta múltiplos idiomas, incluindo o português. No entanto, o Bring! foca-se exclusivamente na gestão de listas, não oferecendo comparação de preços, alertas de variação de preço ou georreferenciação de lojas [?].

O **AnyList** destaca-se pela integração com receitas culinárias, permitindo ao utilizador adicionar automaticamente ingredientes de uma receita à lista de compras. Oferece partilha em tempo real entre utilizadores e suporta organização por categorias. À semelhança do Bring!, não inclui funcionalidades de comparação de preços nem notificações sobre alterações de preço [?].

O **OurGroceries** é uma solução orientada para o contexto familiar, com sincronização automática entre dispositivos. Destaca-se pela simplicidade e rapidez de utilização, mas mantém o mesmo padrão de ausência de informação de preços, scrapers de dados ou integração com sistemas de retalho [?].

Em comum, estas aplicações partilham a limitação de se focarem exclusivamente na gestão de listas sem qualquer mecanismo de integração com dados de preços reais, tornando-se instrumentos de registo e não de apoio à decisão de compra.

3.2 Plataformas de comparação de preços

Existe um conjunto de soluções orientadas especificamente à comparação de preços entre lojas, algumas com foco no mercado português.

O **CompraBem** é uma plataforma portuguesa que permite comparar preços de produtos em múltiplos supermercados, incluindo Continente, Pingo Doce e Auchan. Apresenta o preço atual e o histórico de variações, sendo útil para planeamento de compras. Contudo, a plataforma não oferece funcionalidades de gestão de listas de compras, leitura de código de barras ou notificações de descida de preço, sendo mais uma ferramenta de consulta pontual do que de acompanhamento contínuo [?].

O **Pricena**, popular em alguns mercados, agrega preços de produtos de diferentes revendedores online, permitindo a comparação e a configuração de alertas de preço. Porém, o seu foco está no comércio eletrónico, não cobrindo lojas físicas de supermercados, e não oferece funcionalidades de gestão de listas [?].

Plataformas internacionais como o **Flipp** (disponível nos EUA e Canadá) oferecem uma abordagem mais abrangente, agregando folhetos digitais de supermercados e permitindo criar listas de compras associadas a promoções. No entanto, esta solução não está disponível no mercado português e depende da adesão voluntária das cadeias de retalho para fornecer dados [?].

A limitação transversal a estas plataformas é a falta de integração com ferramentas de gestão de listas e a ausência de funcionalidades de georreferenciação que permitam ao utilizador identificar a loja mais vantajosa na sua proximidade.

3.3 Aplicações próprias de cadeias de supermercados

As principais cadeias de supermercados portuguesas disponibilizam as suas próprias aplicações móveis, que oferecem funcionalidades como listas de compras digitais, cartão de fidelidade e promoções personalizadas.

A aplicação do **Continente** permite ao utilizador criar listas de compras, consultar promoções, aceder ao cartão de fidelidade e efetuar encomendas para entrega ou levantamento. A aplicação é bem desenvolvida e oferece uma experiência integrada, mas está confinada ao ecossistema do Continente, não permitindo comparação com outras insígnias [?].

De forma análoga, a aplicação do **Pingo Doce** oferece funcionalidades de lista de compras, promoções e cartão de fidelidade, mas igualmente restrita ao universo da sua cadeia [?].

A limitação fundamental destas aplicações é precisamente a sua natureza fechada: cada insígnia apresenta apenas os seus próprios preços e promoções, tornando impossível ao utilizador comparar diretamente preços entre cadeias concorrentes a partir de uma única interface.

3.4 Análise comparativa e posicionamento do ShopISEL

A Tabela 3.1 apresenta uma síntese comparativa entre as soluções analisadas e o ShopISEL, considerando as funcionalidades mais relevantes para o utilizador final.

Table 3.1 Comparação entre soluções existentes e o ShopISEL

Funcionalidade	Bring!	CompraBem	Continente	Flipp	ShopISEL
Gestão de listas	✓	–	✓	✓	✓
Comparação de preços	–	✓	–	✓	✓
Múltiplas insígnias	–	✓	–	✓	✓
Alertas de preço	–	–	–	–	✓
Leitura de código de barras	–	–	✓	–	✓
Georreferenciação de lojas	–	–	–	–	✓
Aplicação móvel	✓	–	✓	✓	✓
Aplicação web	–	✓	✓	–	✓
Notificações push	–	–	✓	–	✓
Arquitetura aberta	–	–	–	–	✓

A análise evidencia que nenhuma das soluções existentes combina, numa única plataforma, a gestão de listas de compras com a comparação de preços entre múltiplas insígnias, alertas de variação de preço e georreferenciação de lojas. O ShopISEL posiciona-se precisamente neste espaço, oferecendo uma solução integrada que une estas funcionalidades, disponível tanto em aplicação móvel como em aplicação web.

Adicionalmente, a arquitetura aberta e extensível do ShopISEL, baseada em microserviços com scrapers independentes por insígnia, permite a adição de novas cadeias de retalho sem impacto na estrutura central do sistema, o que constitui uma vantagem significativa face a soluções fechadas ou dependentes da adesão voluntária dos retalhistas.

3.5 Síntese

O levantamento de soluções existentes demonstra que existe uma lacuna clara no mercado para uma plataforma integrada de gestão de compras que combine listas, comparação de preços entre insígnias, alertas e geolocalização. As soluções atuais focam-se geralmente num único domínio funcional, obrigando o utilizador a recorrer a múltiplas aplicações para cobrir as suas necessidades. O ShopISEL procura colmatar esta lacuna, oferecendo um ecossistema digital completo e centrado na experiência de compra do utilizador.

Chapter 4

Trabalho Desenvolvido

Este capítulo apresenta em detalhe a arquitetura, as tecnologias e a implementação dos componentes que constituem o sistema ShopISEL. A solução é composta por múltiplos serviços de backend, duas aplicações cliente (web e móvel), um conjunto de scrapers de dados, um worker de notificações e a infraestrutura de suporte à autenticação, persistência e deployment.

4.1 Visão Geral da Arquitetura

O ShopISEL segue uma arquitetura orientada a serviços, em que cada domínio funcional é tratado por um serviço autónomo com a sua própria base de dados, lógica de negócio e ciclo de deployment. Esta separação permite que cada serviço evolua de forma independente, reduz o acoplamento entre componentes e facilita a escalabilidade horizontal de cada parte do sistema.

A Figura 4.1 apresenta uma visão geral da arquitetura do sistema.

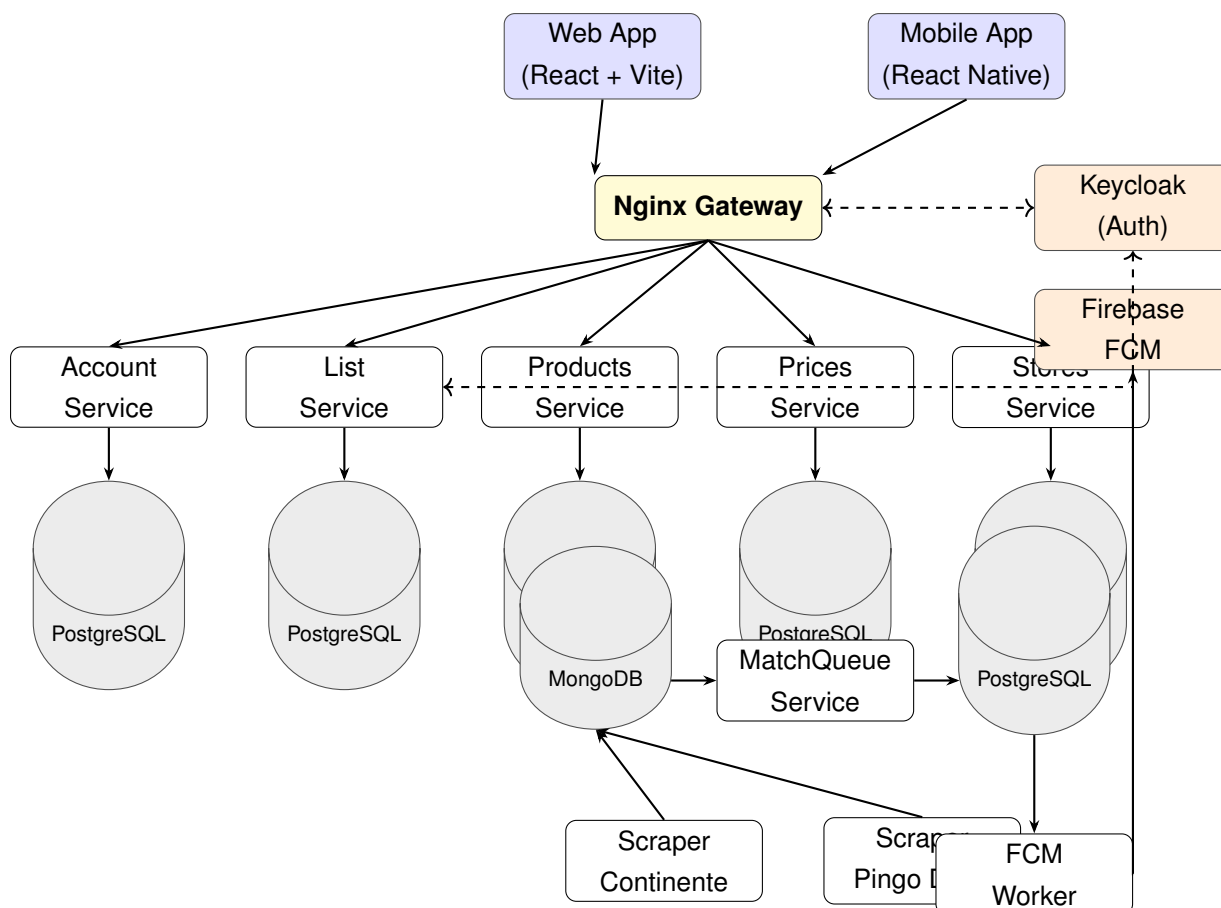


Figure 4.1 Visão geral da arquitetura do sistema ShopISEL

Os pedidos das aplicações cliente chegam ao sistema através de um gateway Nginx, que encaminha cada pedido para o serviço responsável com base no caminho do URL. Os serviços de backend comunicam com as suas respectivas bases de dados PostgreSQL e, no caso dos serviços protegidos, validam os tokens JWT emitidos pelo Keycloak.

A recolha de dados de produtos e preços é feita por scrapers independentes para cada insígnia, que persistem os dados em MongoDB como área de staging. O MatchQueue Service processa estes dados, normalizando-os e integrando-os na base de dados relacional. Por fim, o FCM Notification Worker lê as notificações pendentes e envia alertas para os dispositivos dos utilizadores através do Firebase Cloud Messaging.

4.2 Serviços de Backend

Todos os serviços de backend são desenvolvidos em ASP.NET Core com Minimal APIs, tirando partido da sua leveza, desempenho e boa integração com o ecossistema .NET. Cada serviço dispõe do seu próprio Dockerfile e pipeline de CI/CD, sendo publicado como uma imagem Docker no GitHub Container Registry.

4.2.1 List Service

O List Service é o serviço central do ShopISEL no que diz respeito à interação do utilizador. Gere a criação, consulta, atualização e eliminação de listas de compras, bem como os itens que as compõem.

O modelo de dados do List Service é composto por duas entidades principais:

- **ShoppingListEntity**: representa uma lista de compras, com um identificador único, o identificador do proprietário (`OwnerId`, derivado do claim `sub` do token JWT), o nome da lista e a data de criação.
- **ShoppingListItemEntity**: representa um item da lista, com referência ao identificador do produto (`ProductId`), ao identificador da loja (`StoreId`), à quantidade desejada e ao estado de marcação (`Checked`).

Toda a persistência é assegurada pelo Entity Framework Core com o provider Npgsql para PostgreSQL, com migrações aplicadas automaticamente no arranque do serviço.

A autenticação é obrigatória em todos os endpoints do List Service. O serviço valida tokens JWT emitidos pelo Keycloak, verificando o emissor, a validade temporal e o claim `azp` (authorized party), que deve corresponder a um dos clientes registados (aplicação web ou móvel). Desta forma, cada lista de compras é sempre associada ao utilizador autenticado, e as operações de leitura e escrita são automaticamente filtradas por `OwnerId`, garantindo isolamento entre utilizadores.

Os endpoints expostos são:

- `GET /lists` — devolve todas as listas do utilizador autenticado;
- `POST /lists` — cria uma nova lista com os itens fornecidos;
- `GET /lists/{listId}` — devolve uma lista específica (apenas se pertencer ao utilizador autenticado);
- `PUT /lists/{listId}` — atualiza o nome e os itens de uma lista;
- `DELETE /lists/{listId}` — elimina uma lista.

O serviço inclui também um endpoint de saúde em `GET /health`, utilizado pelos mecanismos de monitorização da plataforma de deploy.

4.2.2 Products Service

O Products Service gere o catálogo de produtos e categorias da plataforma. Não requer autenticação, sendo acessível tanto pelas aplicações cliente como pelos serviços internos.

O modelo de dados contempla duas entidades:

- **ProductEntity**: representa um produto com identificador, nome, código de barras (`Barcode`), URL de imagem e referência à categoria.

- **CategoryEntity**: representa uma categoria hierárquica, com suporte a subcategorias através de uma relação auto-referencial (`ParentCategoryId`).

A hierarquia de categorias permite organizar os produtos de forma estruturada (e.g., Alimentação → Laticínios → Leite), facilitando a navegação e filtragem na interface do utilizador.

Os endpoints de produto suportam filtragem por identificadores, por categoria e por nome, bem como uma pesquisa de produtos relacionados com base em favoritos do utilizador, com parâmetros de limite e distância máxima:

- GET `/products?ids=...` — pesquisa por identificadores;
- GET `/products?categoryId=...` — filtragem por categoria;
- GET `/products?name=...` — pesquisa por nome;
- GET `/products/related?favoriteIds=...` — produtos relacionados com os favoritos.

4.2.3 Prices Service

O Prices Service mantém um registo dos preços de produtos por loja, incluindo suporte a preços de promoção. A entidade **PriceEntity** contém o identificador do produto, o identificador da loja, o preço atual, um preço de promoção opcional (`Sale`) e a data da promoção (`SaleDate`), bem como a data de última atualização.

Este modelo permite ao sistema apresentar ao utilizador não só o preço atual, mas também o histórico de promoções e a variação de preços ao longo do tempo, suportando a funcionalidade de alertas de descida de preço.

4.2.4 Stores Service

O Stores Service gere informação sobre lojas físicas, sendo fundamental para a funcionalidade de georreferenciação. A entidade **StoreEntity** inclui o nome e marca da loja, morada, cidade e coordenadas geográficas (`Latitude`, `Longitude`).

A presença de coordenadas geográficas permite ao sistema calcular a distância entre o utilizador e cada loja, apresentando os preços mais vantajosos em função da proximidade geográfica — um dos elementos diferenciadores do ShopISEL.

4.2.5 Account Service

O Account Service gere o perfil do utilizador autenticado, sincronizando dados provenientes do token JWT emitido pelo Keycloak (nome, email) e permitindo a atualização de preferências do perfil.

O serviço expõe dois endpoints protegidos:

- GET `/accounts/me` — devolve o perfil do utilizador autenticado;

- `PUT /accounts/me` — atualiza as preferências do perfil.

Este serviço é implantado no Fly.io, enquanto os restantes serviços são implantados no Render.com, demonstrando a flexibilidade da arquitetura em termos de plataformas de hosting.

4.2.6 MatchQueue Service

O MatchQueue Service é um componente híbrido que funciona em dois modos de execução: modo API (expõe um endpoint `POST /matchqueue/trigger` que pode ser invocado pelos scrapers após a recolha de dados) e modo worker (processa os dados de staging do MongoDB, normaliza-os e integra-os na base de dados relacional).

Este serviço é responsável pela ponte entre os dados brutos recolhidos pelos scrapers, persistidos em MongoDB, e os dados normalizados que alimentam as restantes funcionalidades do sistema. O modelo de dados do MatchQueue inclui réplicas das entidades de produto, categoria e preço, bem como entidades específicas para a gestão de notificações:

- **FavoriteProductEntity**: regista os produtos favoritos de cada utilizador, utilizados para determinar quais as notificações de descida de preço a enviar;
- **NotificationEnqueueEntity**: fila de notificações pendentes, com referência ao utilizador, ao produto e ao estado de processamento (`Checked`).

O modo worker é ativado através do argumento `-mode worker` na linha de comandos, sendo orquestrado por um GitHub Actions workflow que o dispara após a conclusão dos scrapers.

4.3 Pipeline de Recolha de Dados

A recolha automática de dados de produtos e preços é um dos pilares do ShopISEL, garantindo que a informação apresentada ao utilizador é atual e fiável. A Figura 4.2 ilustra o fluxo de dados desde os scrapers até às notificações.

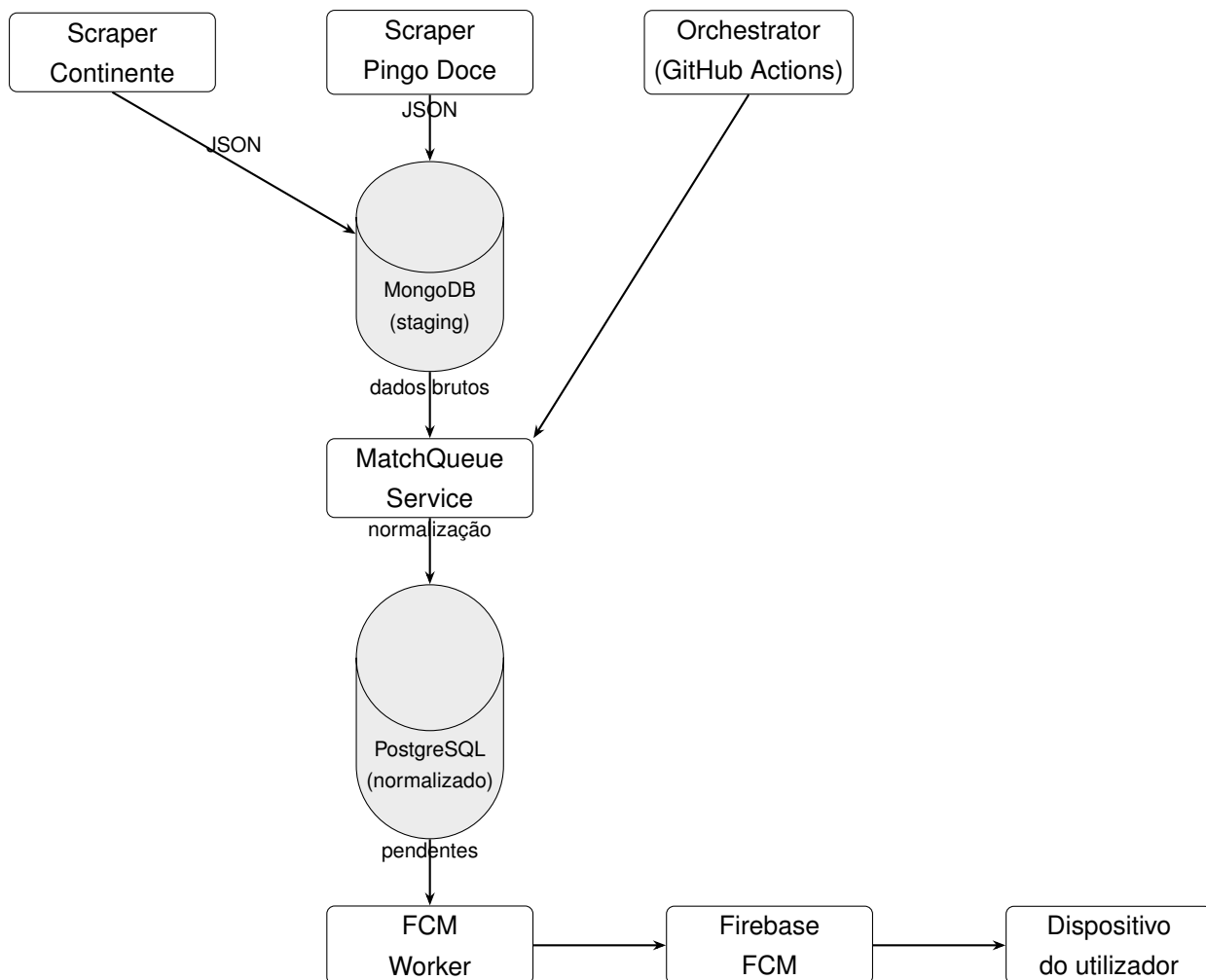


Figure 4.2 Pipeline de recolha de dados e entrega de notificações

4.3.1 Scrapper Continente

O Scrapper Continente é uma aplicação .NET que utiliza o Playwright para navegar automaticamente no website do Continente e extrair informação de produtos por categoria. O Playwright permite controlar um browser real, sendo adequado para páginas com conteúdo renderizado dinamicamente no cliente, como é o caso do website do Continente [?].

O scraper percorre as categorias configuradas, efetua scroll para carregar mais produtos (mecanismo de lazy loading), e extrai para cada produto o nome, preço atual, preço anterior, preço por unidade, disponibilidade, Uniform Resource Locator (URL) da página e URL da imagem. O resultado é exportado para um ficheiro JSON (`continente_products.json`).

A estrutura do scraper divide as responsabilidades em módulos distintos:

- `Scrapers/`: contém a lógica de navegação e extração por categoria;
- `Parsing/`: realiza o parsing do HTML e a normalização de texto (preços, unidades);
- `Models/`: define as estruturas de dados para fontes de categoria e produtos extraídos.

4.3.2 Scraper Pingo Doce

O Scraper Pingo Doce segue a mesma abordagem baseada em Playwright, adaptada à estrutura do website do Pingo Doce. Adicionalmente, suporta dois modos de execução:

- **Modo normal:** efetua o scraping ao vivo e persiste os resultados em MongoDB;
- **Modo -from-json:** processa um ficheiro JSON previamente gerado, evitando pedidos desnecessários ao website.

Os dados são persistidos em MongoDB [?] em duas coleções:

- `scrape_runs`: um documento por execução, identificado por um `run_id` único;
- `scrape_products`: um documento por produto por execução, com um identificador estável baseado no URL externo ou numa combinação de categoria e nome.

Esta estratégia de identificação estável permite ao MatchQueue Service detetar alterações de preço entre execuções consecutivas, comparando os valores pelo mesmo identificador de produto.

Após a conclusão da sincronização com o MongoDB, o scraper pode opcionalmente disparar o processamento no MatchQueue Service através de um pedido `POST /matchqueue/trigger`, configurado por variável de ambiente.

O scraper inclui um workflow de GitHub Actions que o executa de forma agendada (cron), lendo as credenciais de acesso ao MongoDB a partir de segredos do repositório.

4.3.3 Scraper Orchestrator

O Scraper Orchestrator é um repositório de workflows de GitHub Actions responsável por coordenar a execução dos scrapers de múltiplas insígnias. Cada scraper expõe um workflow reutilizável (*reusable workflow*), que pode ser invocado pelo orchestrator através do mecanismo `workflow_call`.

Esta abordagem permite executar os scrapers em paralelo, aguardar a conclusão de todos eles e, de seguida, desencadear o processamento do MatchQueue Service. A adição de um novo scraper para uma insígnia adicional requer apenas a referência ao seu workflow reutilizável no orchestrator, sem necessidade de alterar os scrapers existentes.

4.4 FCM Notification Worker

O FCM Notification Worker é uma aplicação .NET de execução pontual (run-to-completion), concebida para ser invocada periodicamente pelo orchestrator de GitHub Actions. O Firebase Cloud Messaging (Firebase Cloud Messaging (FCM)) [?] é um serviço de entrega de mensagens push mantido pela Google, que permite enviar notificações para dispositivos Android e iOS de forma fiável e escalável. O seu funcionamento segue os seguintes passos:

1. Inicializa o cliente Firebase com as credenciais codificadas em Base64 (provenientes de variável de ambiente);
2. Estabelece ligação à base de dados PostgreSQL do MatchQueue Service;
3. Consulta a tabela `notification_enqueue` para obter notificações pendentes não processadas (`checked = false`), cruzando com a tabela `account_push_tokens` para obter os tokens FCM ativos dos utilizadores;
4. Constrói e envia as mensagens push em lotes de 500 (limite imposto pelo Firebase);
5. Marca as notificações processadas como `checked = true`.

As notificações enviadas informam o utilizador de que o preço de um produto favorito desceu, incluindo nos dados da mensagem o identificador do produto e da notificação, permitindo à aplicação móvel navegar diretamente para o produto relevante.

4.5 Aplicação Web

A aplicação web do ShopISEL é desenvolvida em React com TypeScript e Vite, proporcionando uma interface moderna e responsiva para preparação e análise de compras num ambiente de desktop.

A estrutura da aplicação segue uma organização por responsabilidade:

- `src/api/`: camada de comunicação com os serviços de backend, abstraindo os detalhes de cada pedido HTTP;
- `src/auth/`: integração com o Keycloak para autenticação, gestão do ciclo de vida do token e redirecionamentos;
- `src/app/`: componentes de página e lógica de navegação;
- `src/styles/`: estilos globais e configuração de temas.

A interface recorre a bibliotecas de componentes como Material UI, Radix UI e Tailwind CSS, que permitem construir ecrãs consistentes e acessíveis com menor esforço de implementação. O TypeScript garante a verificação estática dos tipos trocados com as APIs, reduzindo erros em tempo de execução.

A aplicação web é servida em produção através de um servidor Nginx, sendo a imagem Docker publicada automaticamente pelo pipeline de CI/CD.

4.6 Aplicação Móvel

A aplicação móvel é desenvolvida em React Native com o framework Expo, permitindo o desenvolvimento de uma única base de código para Android e iOS. O Expo Router é utilizado para a navegação declarativa entre ecrãs, seguindo uma estrutura baseada em ficheiros análoga ao Next.js.

A estrutura de navegação principal assenta em separadores (*tabs*):

- **Lists** ((*tabs*)/): ecrã principal de gestão de listas de compras;
- **Alerts** (*alerts.tsx*): visualização dos alertas de preço recebidos;
- **Auth** (*auth.tsx*): fluxo de autenticação com Keycloak;
- **Onboarding** (*onboarding.tsx*): ecrã de boas-vindas e configuração inicial.

A organização do código fonte por domínio funcional facilita a manutenção e a extensão da aplicação:

- *src/api/*: chamadas aos serviços de backend;
- *src/auth/*: gestão do token de autenticação com Expo Secure Store;
- *src/push/*: integração com Firebase Cloud Messaging para receção de notificações;
- *src/location/*: serviços de georreferenciação para apresentação de lojas próximas;
- *src/favorites/*: gestão dos produtos favoritos do utilizador;
- *src/i18n/*: suporte a múltiplos idiomas (internacionalização);
- *src/theme/*: sistema de temas e estilos globais.

A leitura de códigos de barras é realizada com o módulo Expo Camera, permitindo ao utilizador identificar rapidamente um produto durante a compra. O armazenamento seguro dos tokens de autenticação utiliza o Expo Secure Store, que persiste dados de forma cifrada no dispositivo.

O NativeWind é utilizado para estilização, aplicando as utilidades do Tailwind CSS ao contexto React Native [?], garantindo consistência visual com a aplicação web.

4.7 Autenticação e Autorização

A autenticação e autorização no ShopISEL é centralizada no Keycloak, uma plataforma de gestão de identidade e acesso que implementa os protocolos OpenID Connect (OIDC) e OAuth 2.0.

O fluxo de autenticação segue o padrão Authorization Code Flow com PKCE (Proof Key for Code Exchange), adequado tanto para aplicações web como para aplicações móveis. O utilizador é redirecionado para o servidor Keycloak para introduzir as suas credenciais; após autenticação bem-sucedida, é emitido um token de acesso JWT que é enviado em cada pedido aos serviços protegidos.

Os serviços que requerem autenticação (List Service e Account Service) validam o token da seguinte forma:

1. Verificam que o emissor (*iss*) corresponde ao realm Keycloak configurado;

2. Verificam que o token não expirou (`exp`);
3. Verificam que a chave de assinatura corresponde à chave pública do Keycloak (obtida via OIDC Discovery);
4. Verificam que o claim `azp` (authorized party) corresponde a um dos clientes autorizados (web, mobile).

Esta abordagem evita que cada serviço implemente autenticação própria, centraliza a gestão de utilizadores no Keycloak e permite que novos serviços se integrem facilmente no ecossistema de autenticação existente.

4.8 Gateway e Infraestrutura de Deployment

O acesso às APIs dos serviços de backend é feito através de um gateway Nginx, que atua como ponto de entrada único para as aplicações cliente. O Nginx encaminha os pedidos para o serviço correto com base no caminho do URL:

- `/lists/...` → List Service;
- `/products/...` → Products Service;
- `/prices/...` → Prices Service;
- `/stores/...` → Stores Service;
- `/` → Aplicação Web (frontend).

Esta arquitetura de gateway simplifica a configuração das aplicações cliente, que comunicam com um único endpoint base, e permite adicionar funcionalidades transversais como rate limiting, logging centralizado ou cache HTTP sem modificar os serviços individuais.

Todo o sistema é containerizado com Docker. Cada serviço dispõe de um Dockerfile que produz uma imagem mínima e otimizada para produção. Os serviços são implantados no Render.com, com o Account Service a ser implantado no Fly.io, tirando partido das garantias de disponibilidade e escalabilidade destas plataformas.

A configuração de deployment está declarada no ficheiro `render.yaml`, que especifica os serviços, as imagens Docker correspondentes e as variáveis de ambiente necessárias. As imagens são publicadas automaticamente no GitHub Container Registry pelos pipelines de CI/CD, e o Render.com deteta novas versões e realiza o deploy de forma automática.

Chapter 5

Processo de Desenvolvimento e Integração Contínua

Este capítulo descreve o processo de desenvolvimento adotado no projeto ShopISEL, incluindo a organização dos repositórios, as práticas de controlo de versões, as pipelines de integração e entrega contínua (CI/CD) e a estratégia de deployment automatizado.

5.1 Organização do Projeto em Repositórios

O ShopISEL adota uma estrutura multi-repositório, em que cada serviço ou componente principal reside num repositório Git independente. Esta abordagem permite que cada equipa ou componente evolua autonomamente, com o seu próprio ciclo de testes, construção e publicação, sem introduzir dependências de build entre serviços distintos.

Os repositórios principais do projeto são:

- `list-service` — serviço de listas de compras;
- `products-service` — serviço de produtos e categorias;
- `prices-service` — serviço de preços;
- `stores-service` — serviço de lojas e georreferenciação;
- `account-service` — serviço de perfil de utilizador;
- `matchqueue-service` — serviço de correspondência e normalização de dados;
- `fcm-notification-worker` — worker de notificações push;
- `scraper-continente` — scraper do website Continente;
- `scraper-pingodoce` — scraper do website Pingo Doce;
- `scraper-orchestrator` — orchestrator dos scrapers via GitHub Actions;
- `frontend` — aplicação web React;

- `frontend-mobile` — aplicação móvel React Native / Expo;
- `deploy` — configuração de infraestrutura e gateway Nginx;
- `keycloak` — configuração do servidor Keycloak.

5.2 Controlo de Versões e Estratégia de Branches

O controlo de versões é assegurado pelo Git, com os repositórios alojados no GitHub. O branch principal de cada repositório é o `main`, que representa sempre um estado estável e implantável do serviço. O desenvolvimento de novas funcionalidades é realizado em branches separados, que são integrados no `main` através de Pull Requests após revisão de código.

Esta prática permite que a pipeline de CI/CD, configurada no branch `main`, seja ativada apenas quando o código está pronto para integração, garantindo que apenas código revisto e testado é publicado e implantado.

5.3 Pipelines de Integração e Entrega Contínua

Cada repositório de backend inclui uma pipeline de CI/CD implementada com GitHub Actions. As pipelines são ativadas automaticamente em cada push para o branch `main` e seguem, tipicamente, as seguintes etapas:

1. **Build e teste:** o código é compilado e os testes automatizados são executados. Esta etapa garante que o código integrado compila sem erros e que os testes de integração passam antes de avançar para as etapas seguintes.
2. **Análise de qualidade:** em alguns serviços, é realizada uma análise estática de código através do SonarCloud, que reporta métricas de qualidade, vulnerabilidades e código duplicado.
3. **Construção da imagem Docker:** a imagem Docker do serviço é construída com base no Dockerfile do repositório.
4. **Publicação da imagem:** a imagem construída é publicada no GitHub Container Registry (`ghcr.io/shopisel/<service-name>:latest`), ficando disponível para o processo de deployment.
5. **Deploy automático:** as plataformas de hosting (Render.com e Fly.io) são configuradas para detetar novas versões da imagem e realizar o deployment de forma automática.

Para os scrapers, as pipelines incluem uma etapa adicional de execução agendada por cron, que dispara automaticamente o processo de recolha de dados em intervalos regulares, sem necessidade de intervenção manual.

A Figura 5.1 ilustra o fluxo geral de CI/CD para os serviços de backend.

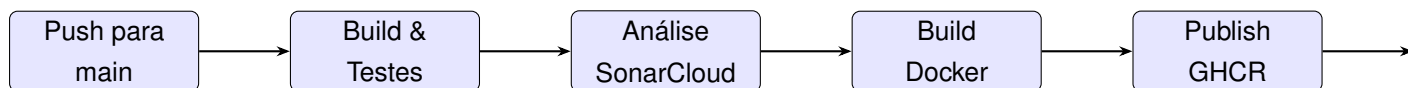


Figure 5.1 Fluxo de CI/CD para os serviços de backend

5.4 Reusable Workflows e Orquestração de Scrapers

Um aspecto relevante do processo de desenvolvimento é a utilização de GitHub Actions reusable workflows para a orquestração dos scrapers. Cada repositório de scraper expõe o seu processo de execução como um workflow reutilizável (`workflow_call`), que pode ser invocado por repositórios externos.

O repositório `scraper-orchestrator` contém um workflow que:

1. Invoca em paralelo os workflows reutilizáveis de cada scraper;
2. Aguarda a conclusão de todos os scrapers (`needs: [...]`);
3. Aciona o MatchQueue Service para processar os dados recolhidos.

Esta arquitetura de orquestração permite adicionar suporte a uma nova insígnia simplesmente criando um novo repositório de scraper com um workflow reutilizável e referenciando-o no orchestrator, sem modificar nenhum dos scrapers existentes.

5.5 Ferramentas de Suporte ao Desenvolvimento

O desenvolvimento do ShopISEL recorreu a um conjunto de ferramentas complementares que contribuíram para a qualidade e eficiência do processo:

- **GitHub Copilot:** ferramenta de assistência à escrita de código baseada em modelos de linguagem de grande dimensão (*Large Language Models*), utilizada para sugestões de código, geração de testes e validação de lógica de negócio.
- **SonarCloud:** plataforma de análise estática integrada nas pipelines de CI/CD, que reporta métricas de qualidade de código, vulnerabilidades de segurança e cobertura de testes.
- **Docker:** utilizado para containerizar todos os serviços, garantindo reprodutibilidade dos ambientes de desenvolvimento, teste e produção.
- **GitHub Container Registry:** repositório de imagens Docker integrado no GitHub, utilizado para publicar e versionar as imagens de cada serviço.
- **Render.com e Fly.io:** plataformas de cloud hosting utilizadas para o deployment dos serviços, com suporte nativo a imagens Docker e deployment automático por webhook.

5.6 Gestão de Configuração e Segredos

A configuração de cada serviço é injetada por variáveis de ambiente, seguindo o princípio dos 12-factor apps. Isto permite que a mesma imagem Docker seja executada em diferentes ambientes (desenvolvimento, staging, produção) com configurações distintas, sem necessidade de recompilação.

Os segredos sensíveis (cadeias de ligação a bases de dados, credenciais do Keycloak, credenciais Firebase, tokens de acesso ao MongoDB) são geridos como GitHub Secrets nos repositórios correspondentes, sendo injetados nas pipelines de CI/CD e nos ambientes de produção sem exposição no código fonte.

Esta prática é particularmente relevante para os scrapers, cujos workflows de GitHub Actions necessitam de acesso às credenciais do MongoDB e às configurações do MatchQueue Service para funcionar corretamente.

Chapter 6

Testes e Avaliação

A fase de testes e avaliação é parte integrante do processo de desenvolvimento, permitindo validar o comportamento do sistema, identificar regressões e garantir a conformidade dos contratos das APIs. Este capítulo descreve a estratégia de testes adotada no ShopISEL, os mecanismos de isolamento utilizados e os resultados obtidos.

6.1 Estratégia de Testes

O ShopISEL adota uma abordagem de testes de integração de ponta a ponta (end-to-end) ao nível de cada serviço, em detrimento de testes unitários isolados. Esta opção justifica-se pela natureza dos serviços: dado que a maioria da lógica de negócio se manifesta na interação entre endpoints HTTP, persistência em base de dados e validação de autenticação, os testes de integração oferecem um nível de confiança superior sobre o comportamento real do sistema.

Os testes são implementados com o framework xUnit [?], amplamente adotado no ecossistema .NET, e recorrem à classe `WebApplicationFactory<Program>` da ASP.NET Core para inicializar o serviço em memória, sem necessidade de servidores externos [?].

Os serviços com cobertura de testes de integração são:

- **List Service** (`ListService.Tests`);
- **Products Service** (`ProductService.Tests`);
- **Prices Service** (`PricesService.Tests`).

6.2 Infraestrutura de Testes

Para que os testes de integração possam ser executados de forma isolada e reproduzível, sem depender de serviços externos como PostgreSQL em produção ou Keycloak, são utilizados dois mecanismos de substituição:

6.2.1 Base de dados em memória com SQLite

A classe `ListServiceApiFactory` (e equivalentes nos outros serviços) substitui o contexto de base de dados da aplicação por uma instância SQLite com ficheiro temporário, criada para cada execução de testes e eliminada no final. Esta abordagem garante que:

- Cada execução de testes parte de um estado limpo e previsível;
- Não é necessário provisionar uma instância de PostgreSQL para executar os testes localmente ou nas pipelines de CI/CD;
- Os testes são completamente deterministas e não interferem entre si.

6.2.2 Handler de autenticação de teste

Dado que os endpoints protegidos requerem um token JWT válido emitido pelo Keycloak, os testes substituem o esquema de autenticação real por um handler personalizado (`TestAuthHandler`). Este handler autentica automaticamente cada pedido HTTP de teste com um utilizador fixo, permitindo testar o comportamento dos endpoints sem necessidade de uma instância Keycloak em execução.

Para testar cenários de isolamento entre utilizadores, é possível criar clientes HTTP adicionais com identidades distintas, modificando o header de utilizador de teste (`X-Test-User`). Esta técnica é utilizada nos testes de isolamento descritos na Secção 6.3.

6.3 Casos de Teste do List Service

O conjunto de testes do List Service cobre os principais fluxos de negócio e requisitos de segurança do serviço.

6.3.1 Fluxo CRUD completo

O teste `ListsCrudFlow_WorksEndToEnd` valida o ciclo de vida completo de uma lista de compras:

1. **Criação:** submete um pedido `POST /lists` com nome e dois itens. Verifica que a resposta tem código HTTP 201, que o identificador da lista começa com o prefixo `list_` e que os dois itens foram criados com identificadores positivos.
2. **Consulta:** faz `GET /lists/{id}` e verifica que o nome da lista corresponde ao submetido.
3. **Atualização:** submete `PUT /lists/{id}` com novo nome e um único item diferente. Verifica que a resposta reflete as alterações corretamente.
4. **Eliminação:** faz `DELETE /lists/{id}` e verifica que a resposta é HTTP 204. Subsequentemente, um `GET /lists/{id}` deve retornar HTTP 404.

6.3.2 Isolamento por proprietário

O teste `ListOwnerIsolation_OtherUserCannotAccessList` verifica que um utilizador não consegue aceder às listas de outro utilizador:

1. O utilizador A cria uma lista;
2. Um cliente HTTP configurado com a identidade do utilizador B tenta obter e eliminar a lista;
3. Verifica-se que ambas as operações retornam HTTP 404, confirmando que o isolamento por `OwnerId` funciona corretamente.

Este teste é fundamental para garantir que a privacidade dos dados do utilizador é respeitada, mesmo perante pedidos autenticados de outros utilizadores.

6.3.3 Filtragem no GET /lists

O teste `GetAll_ReturnsOnlyListsFromAuthenticatedUser` verifica que o endpoint `GET /lists` devolve apenas as listas do utilizador autenticado:

1. O utilizador A e o utilizador B criam, cada um, uma lista;
2. O utilizador A executa `GET /lists` e verifica que a resposta contém a sua lista mas não a do utilizador B.

6.4 Execução dos Testes nas Pipelines de CI/CD

Os testes são integrados nas pipelines de CI/CD de cada serviço. Na etapa de *build e teste*, o comando `dotnet test` é executado, correndo todos os testes de integração do projeto. Se algum teste falhar, a pipeline é interrompida e o código não avança para a etapa de construção da imagem Docker, impedindo que código com regressões seja publicado ou implantado.

Esta integração garante que qualquer alteração ao código que quebre um contrato de API ou introduza uma regressão de comportamento é detetada automaticamente, antes de chegar ao ambiente de produção.

6.5 Avaliação Funcional

Para além dos testes automatizados, o sistema foi sujeito a validação funcional manual, que permitiu verificar o comportamento integrado de todos os componentes em conjunto — aspeto que os testes de integração por serviço não cobrem, dado que cada serviço é testado de forma isolada.

A validação funcional incluiu os seguintes cenários:

- **Registo e autenticação:** criação de conta no Keycloak, login a partir da aplicação web e da aplicação móvel, renovação de token e logout;
- **Gestão de listas:** criação, edição e eliminação de listas de compras na aplicação web e na aplicação móvel, com verificação da sincronização entre dispositivos;
- **Pesquisa de produtos:** pesquisa por nome e por categoria, consulta de preços por loja;
- **Leitura de código de barras:** identificação de produtos através da câmara na aplicação móvel;
- **Georreferenciação:** apresentação de lojas próximas com base na localização do dispositivo;
- **Notificações push:** registo do token FCM no arranque da aplicação móvel e receção de alertas de descida de preço;
- **Pipeline de scraping:** execução dos scrapers, verificação dos dados no MongoDB, processamento pelo MatchQueue Service e consulta dos preços atualizados na aplicação.

A validação funcional confirmou que o sistema se comporta de acordo com os requisitos definidos, com tempos de resposta adequados às necessidades de utilização em contexto real de compra.

Chapter 7

Conclusão

Este capítulo apresenta uma síntese do trabalho desenvolvido no âmbito do projeto ShopISEL, reflete sobre os principais desafios encontrados e identifica possibilidades de evolução futura da plataforma.

7.1 Síntese do Trabalho Desenvolvido

O ShopISEL partiu de uma necessidade concreta dos consumidores: o planeamento eficiente de compras num contexto de múltiplas insígnias, preços variáveis e informação dispersa. A resposta desenvolvida é uma plataforma digital integrada, composta por uma aplicação móvel, uma aplicação web e uma infraestrutura de backend baseada em microserviços.

O sistema oferece ao utilizador a possibilidade de criar e gerir listas de compras, comparar preços do mesmo produto entre diferentes lojas e localizações, receber alertas quando o preço de um produto favorito desce e identificar produtos através da leitura de códigos de barras. A georreferenciação das lojas permite apresentar opções ordenadas por proximidade geográfica, tornando a informação contextualmente relevante.

Do ponto de vista técnico, o projeto demonstra como uma aplicação aparentemente simples — uma lista de compras — pode evoluir para um ecossistema digital completo, com desafios técnicos significativos ao nível da recolha e normalização de dados heterogéneos, da integração de múltiplos serviços, da autenticação centralizada e da entrega de notificações em tempo real.

Os principais componentes desenvolvidos foram:

- Cinco serviços de backend em ASP.NET Core (listas, produtos, preços, lojas, contas), cada um com persistência relacional em PostgreSQL;
- Um serviço híbrido (MatchQueue) que faz a ponte entre dados de scraping armazenados em MongoDB e a base de dados relacional normalizada;
- Dois scrapers baseados em Playwright para recolha automática de dados do Continente e do Pingo Doce;

- Um orchestrator de scrapers baseado em GitHub Actions reusable workflows;
- Um worker de notificações que integra a base de dados com o Firebase Cloud Messaging;
- Uma aplicação web em React com TypeScript e Vite;
- Uma aplicação móvel em React Native com Expo, com suporte a leitura de código de barras, georreferenciação, notificações push e internacionalização;
- Um gateway Nginx como ponto de entrada unificado para as aplicações cliente;
- Pipelines de CI/CD com GitHub Actions para todos os serviços, com publicação automática no GitHub Container Registry e deployment nas plataformas Render.com e Fly.io.

7.2 Desafios Encontrados

O desenvolvimento do ShopISEL apresentou desafios técnicos relevantes em várias frentes.

Recolha e normalização de dados. A extração de dados dos websites do Continente e do Pingo Doce revelou-se um processo frágil, dado que as estruturas HTML das páginas podem mudar sem aviso. O uso do Playwright permitiu lidar com páginas de renderização dinâmica, mas a manutenção dos scrapers exige atenção contínua às alterações dos websites fontes. A normalização dos dados provenientes de fontes heterogéneas, com formatos e nomenclaturas distintos, exigiu a criação de uma camada de matching (MatchQueue Service) que identificasse o mesmo produto em diferentes insígnias.

Isolamento de autenticação em testes. A integração com o Keycloak é essencial para a segurança do sistema, mas representa um ponto de dependência externa nos testes. A solução encontrada, com a criação de um `TestAuthHandler` que simula a autenticação em contexto de teste, permite executar testes de integração completos sem necessidade de uma instância Keycloak externa.

Arquitetura multi-repositório. A gestão de múltiplos repositórios com ciclos de desenvolvimento independentes requer disciplina no versionamento, na comunicação de contratos entre serviços e na coordenação de alterações que afetam múltiplos componentes. A utilização de GitHub Actions e de ficheiros de configuração declarativos (`render.yaml`) contribuiu para mitigar esta complexidade.

Georreferenciação e privacidade. A funcionalidade de apresentação de lojas próximas requer acesso à localização do dispositivo, o que levanta considerações de privacidade que devem ser comunicadas ao utilizador de forma clara. A implementação respeita as boas práticas de pedido explícito de permissões em tempo de execução, em conformidade com as diretrizes das plataformas Android e iOS.

7.3 Trabalho Futuro

O ShopISEL foi desenvolvido com uma arquitetura extensível, que facilita a adição de novas funcionalidades e o alargamento do âmbito do sistema. As principais direções de evolução identificadas são:

Suporte a novas insígnias. A adição de novos scrapers para outras cadeias de supermercados portuguesas (e.g., Auchan, Lidl, Intermarché) é a extensão mais natural do sistema, não requerendo alterações à infraestrutura central. Cada nova insígnia corresponde a um novo repositório de scraper com um workflow reutilizável registado no orchestrator.

Histórico de preços e análise de tendências. Atualmente, o sistema mantém o preço mais recente de cada produto por loja. A extensão do modelo de dados para preservar o histórico completo de variações permitiria apresentar ao utilizador gráficos de evolução de preço e detetar padrões sazonais.

Recomendações personalizadas. Com base nos produtos favoritos, no histórico de listas e nas preferências do utilizador, o sistema poderia oferecer sugestões de produtos ou de lojas mais vantajosas, incorporando lógica de recomendação personalizada.

Partilha de listas. A funcionalidade de partilha de listas de compras entre utilizadores (e.g., membros de uma família) é uma extensão natural do modelo atual, que requer a adição de um mecanismo de partilha e controlo de acesso ao nível do List Service.

Digitalização de talões. A leitura e análise de talões de compra por OCR permitiria ao utilizador registar automaticamente o que comprou, o preço pago e a loja, construindo um histórico de compras pessoal e alimentando o sistema com dados de preços reais praticados.

Desenvolvimento White Label. A arquitetura modular da plataforma e a capacidade de customização das aplicações cliente através de pipelines de CI/CD dedicadas abre a possibilidade de oferecer o sistema como plataforma white label, adaptável a diferentes contextos de retalho ou parceiros comerciais.

7.4 Considerações Finais

O projeto ShopISEL demonstra que é possível, com tecnologias modernas e práticas de engenharia de software bem estabelecidas, construir uma plataforma digital de qualidade que responde a uma necessidade real dos consumidores. A combinação de uma arquitetura orientada a serviços, recolha automática de dados, autenticação centralizada, notificações push e interfaces nativas para web e mobile resulta num sistema coerente, extensível e pronto para evolução.

A adoção de práticas de CI/CD, containerização e deployment declarativo garante que o sistema pode ser evoluído e entregue com confiança, mantendo a qualidade e a consistência ao longo do ciclo de vida do produto.

Em suma, o ShopISEL representa um primeiro passo sólido para uma plataforma de apoio à decisão de compra que, com as evoluções descritas, tem potencial para se tornar uma ferramenta genuinamente útil no quotidiano dos consumidores portugueses.

